

Software Engineering

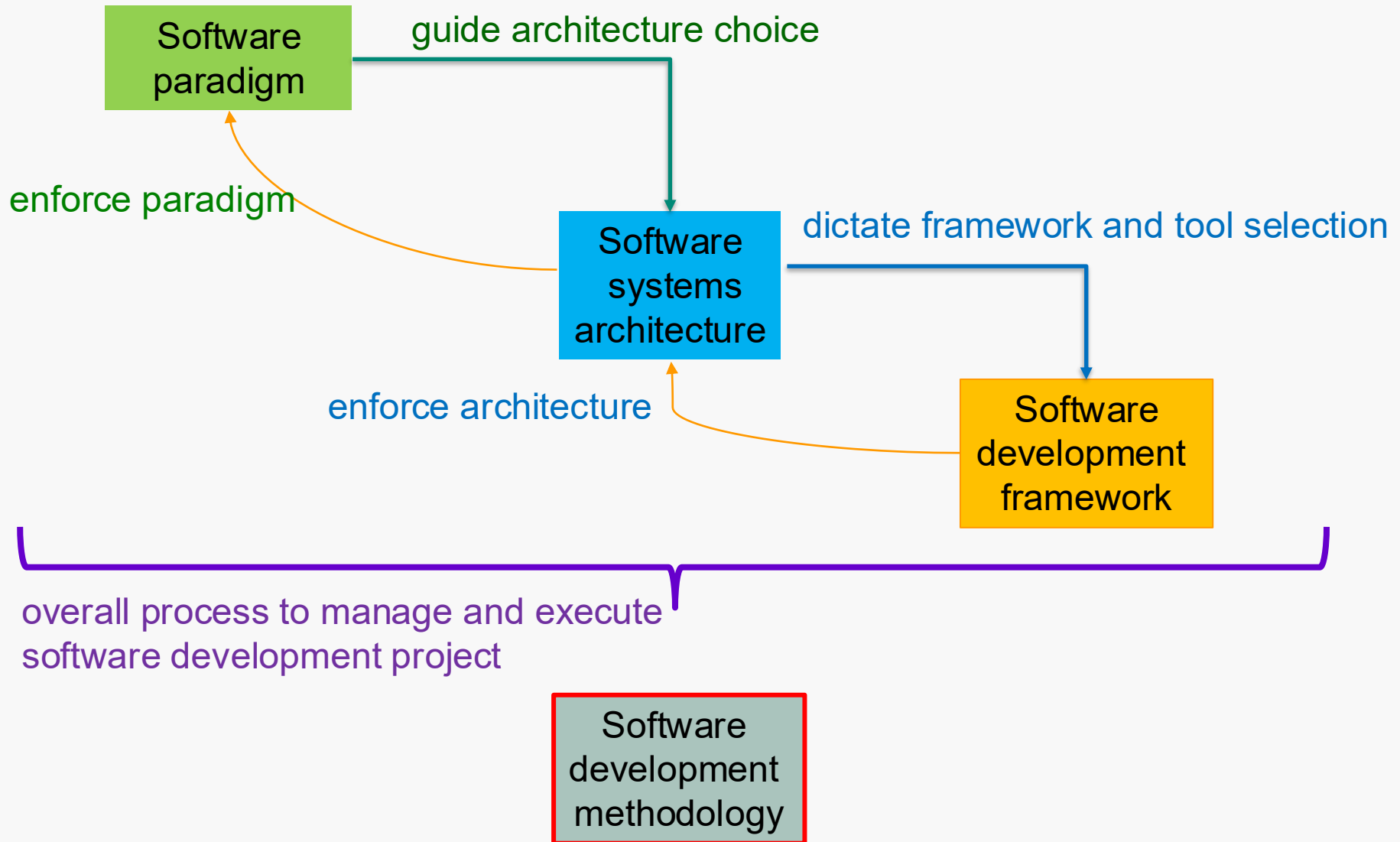
- DevOps engineering for developing and delivering software at a rapid changing pace

Dr. Lily (Dr. Nisha Singh)

Learning Objectives

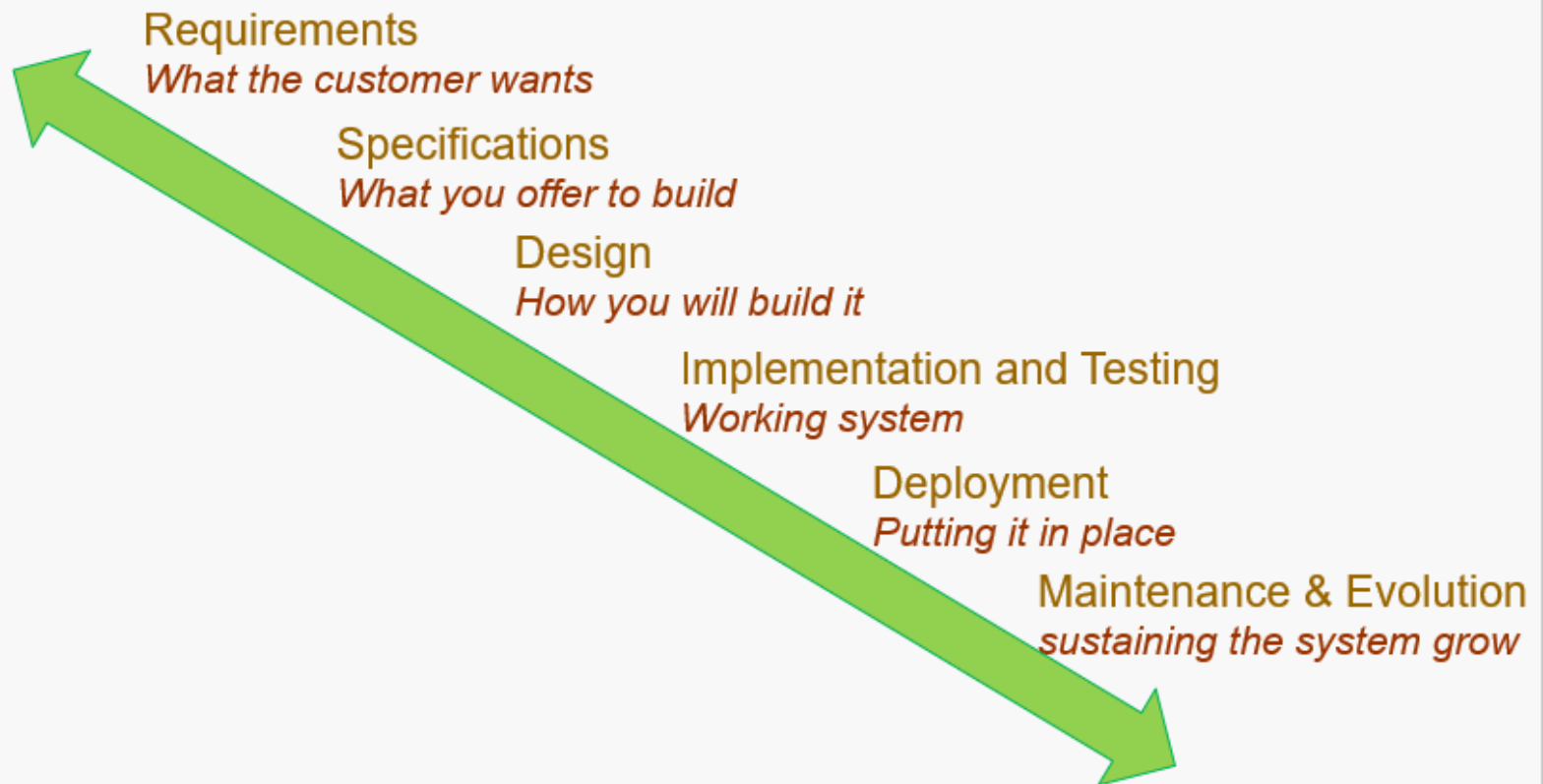
- ❑ What is DevOps?
- ❑ Explore DevOps Lifecycle and Associated Tools
- ❑ CI/CD Pipeline
- ❑ DevOps Critical Success Factors
- ❑ DevOps Challenges
- ❑ Netflix Case Study
- ❑ AI in DevOps

SE Conceptual Flow



Transitional SE Methodologies

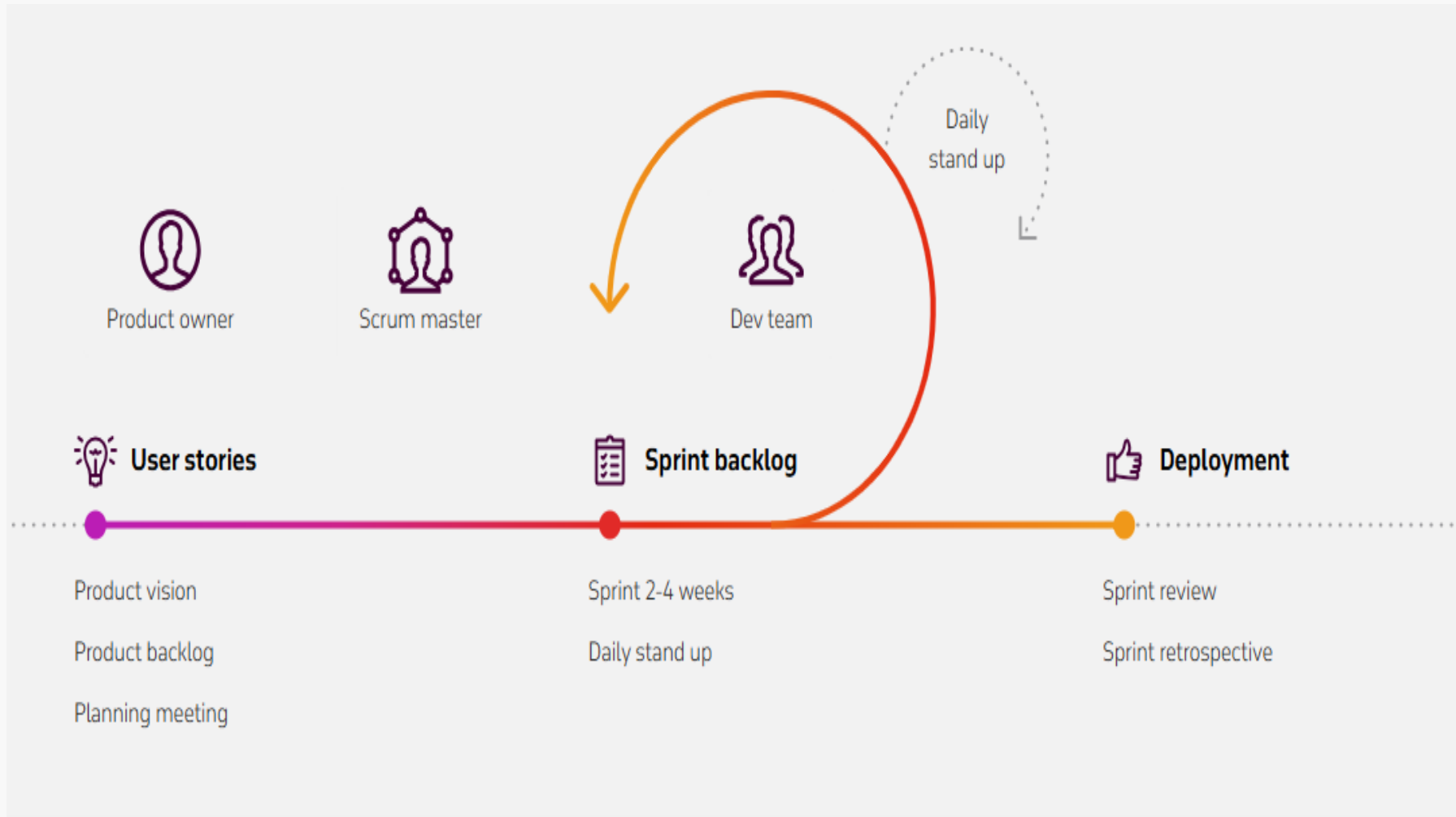
- ❑ Software engineering methodology refers to a structured framework used to plan, manage, and execute the software development process.
- ❑ Waterfall → Agile → DevOps



Challenges of Traditional SDLC

- ❑ **Traditional SDLC** worked well for small, well-defined projects, but struggles to meet the pace and complexity of modern software development
- ❑ **Lack of Flexibility** – Once development begins, incorporating changes becomes difficult.
- ❑ **Delayed Feedback** – Issues are detected late since testing occurs much later in the process.
- ❑ **Isolated Teams** – Developers, testers, and operations often work separately, leading to poor communication and coordination.
- ❑ **Difficulty in Adapting to Change** – Changing requirements are costly to implement and may even force teams to restart projects.

Agile Methodology



Scrum Agile Model

Evolution of DevOps

Large organisations often have multiple teams with different goals and priorities.

Development teams:

- Focus on creating software.
- Face increasing demand for rapid development pace

Operations teams:

- Ensure software runs smoothly in the customer environment.
- Prefer fewer changes to minimize production failures.

Software Company



Development Team



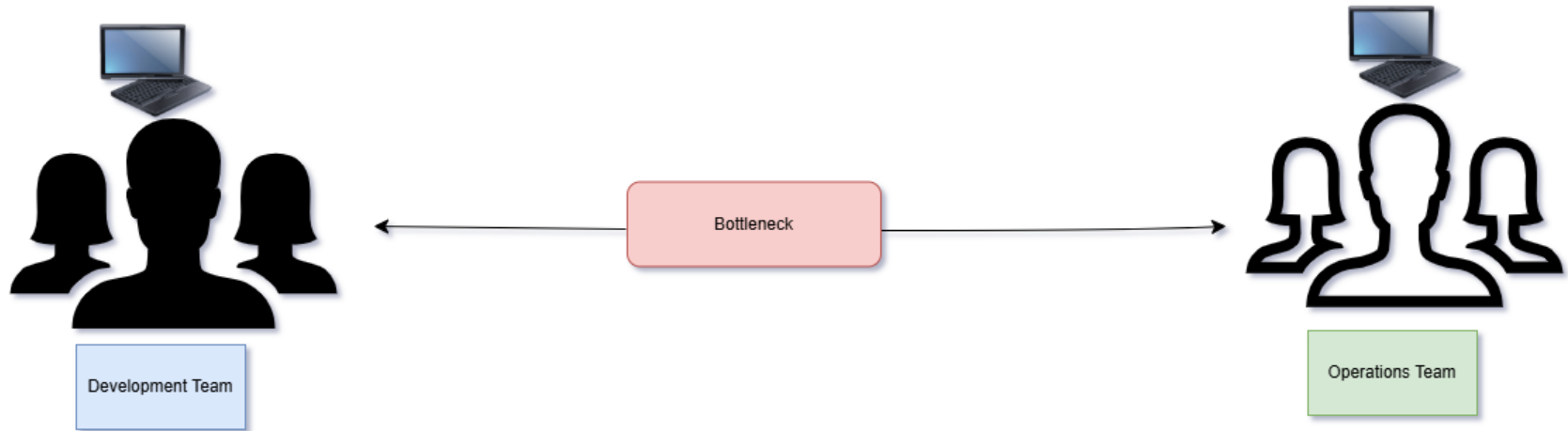
Operations Team

Faster development pace creates **bottlenecks** between Development and Operations teams

Why DevOps ?

The code runs fine on the developer's system and says, "It's not my fault."

The operations team marked this code faulty and used to forward this feedback to the development team.

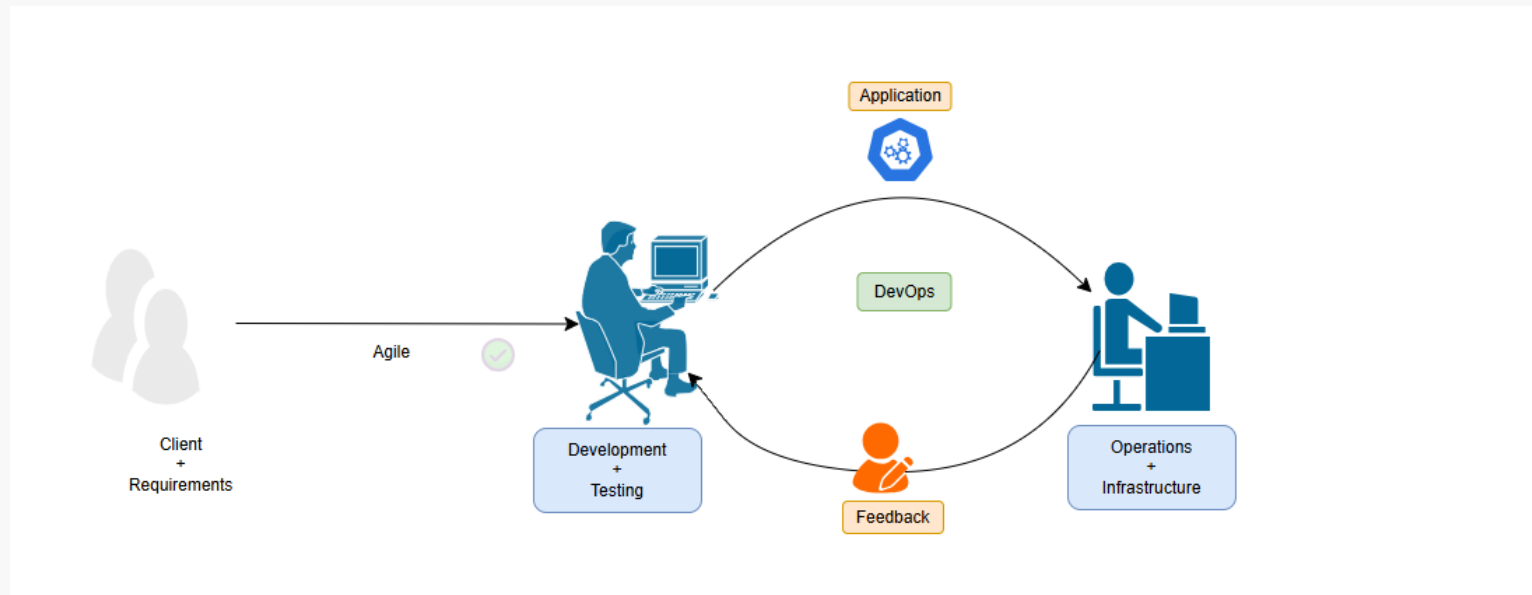


These problems have caused release cycles to extend for months, negatively affecting the organisation's competitiveness.

Solution to this problem: DevOps

What is DevOps

- ❑ DevOps addresses the gap between Developers and Operations.
- ❑ DevOps is coined from the names of two core teams, development and operations.



Agile addressed the gap between clients and developers

The operations team will monitor the application and provide relevant feedback to developers

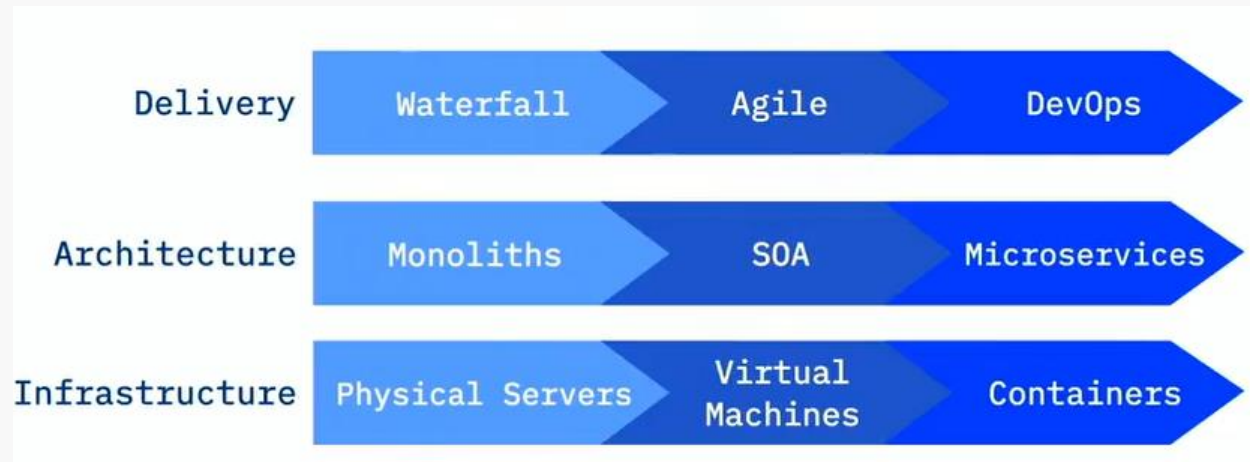
Definition of DevOps

- **Patrick Debois** – consultant passionate about agile development.
- Noticed a **mismatch between IT operations and software development teams**.
- **2008**: Shared issues regarding software development and team discrepancies with agile practitioner **Andrew Schafer**.
- **2009**: Organized the first “**DevOpsDays**” conference in Belgium.
- The term “**DevOps**” was used for the first time.

“The term (development and operations) is an extension of agile development environments that aims to enhance the process of software delivery as a whole.”

–Patrick Debois, 2009

Application Evolution



DevOps Engineering

- ❑ Uses CI/CD pipelines to ensure rapid and reliable delivery.
- ❑ Encourages collaboration, automation, and continuous feedback

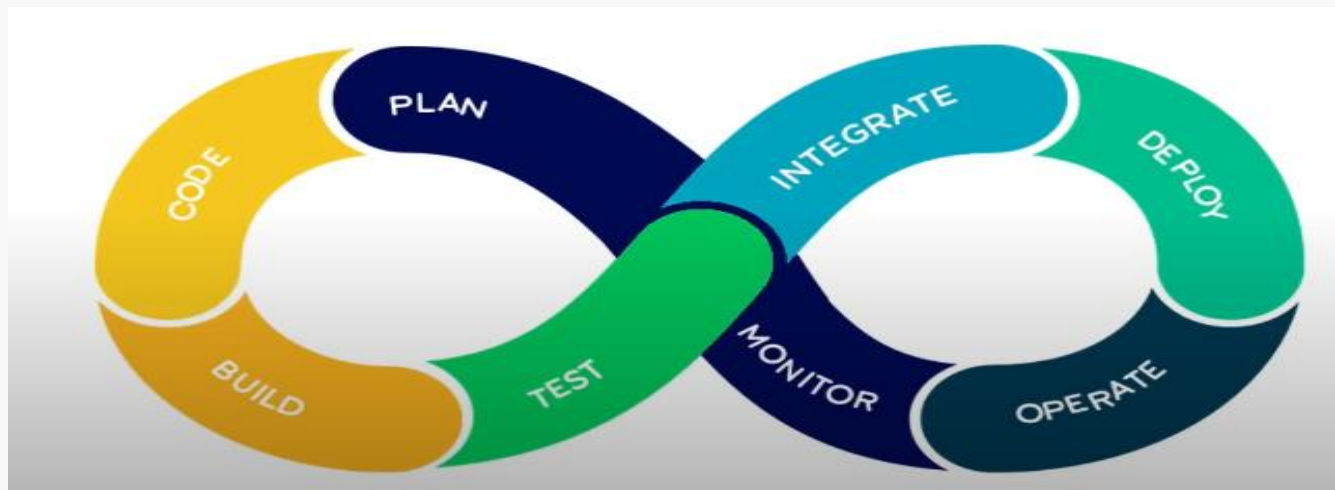
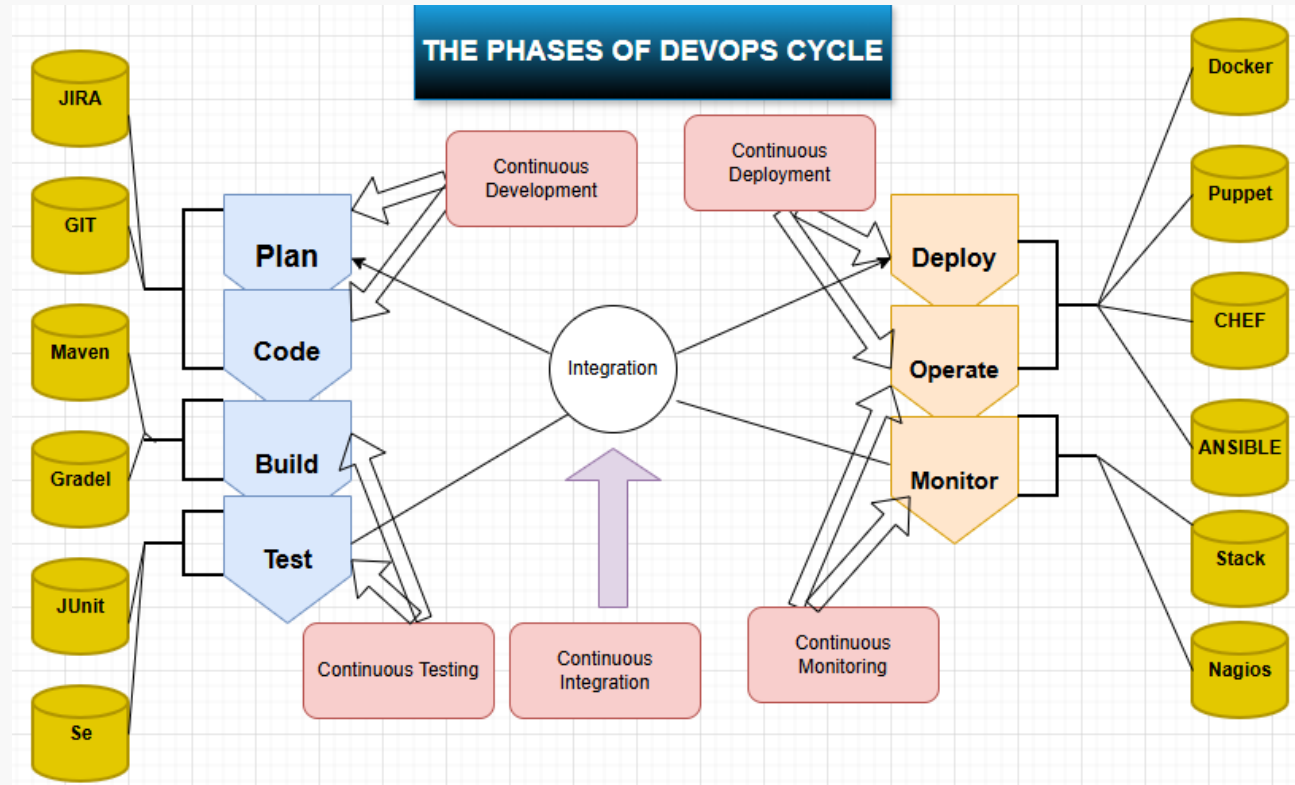


Image Src: [devop infinity symbol explanation - Search Images](#)

The DevOps lifecycle — a continuous feedback loop

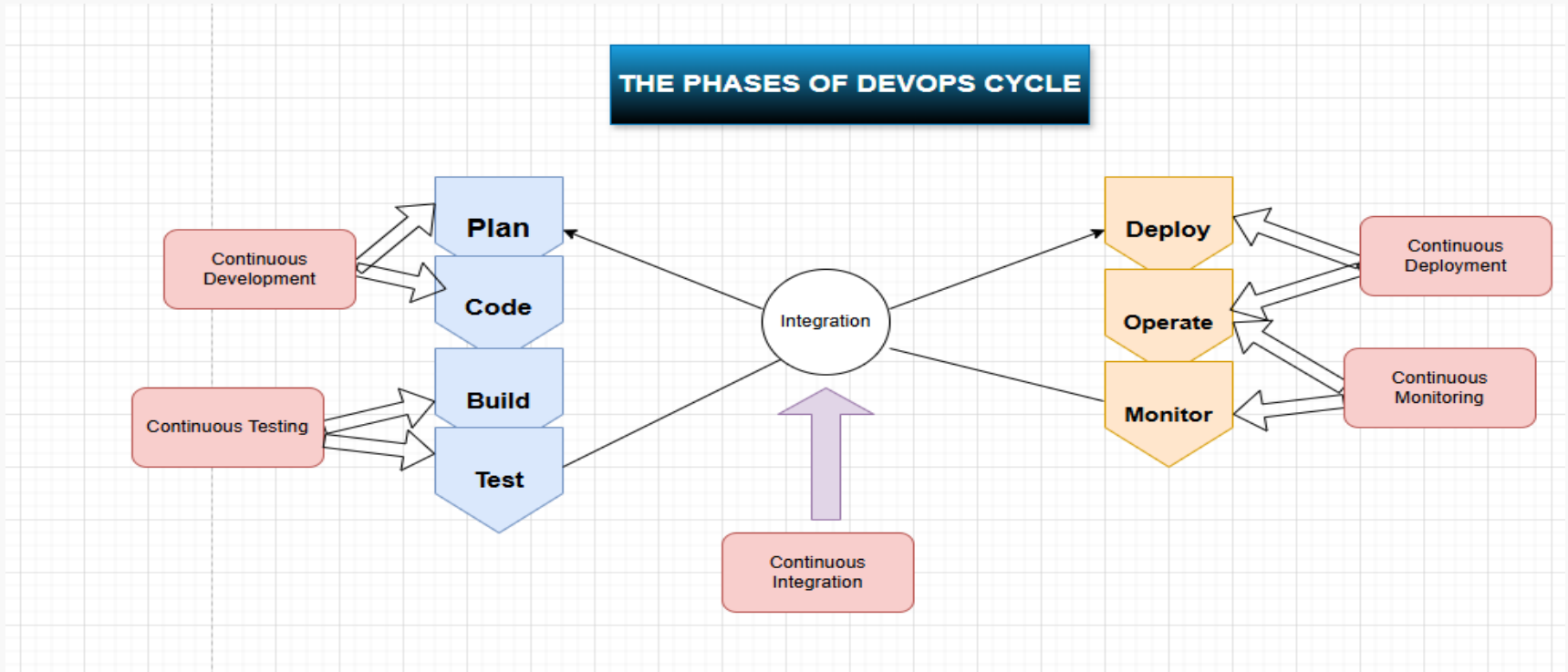
Tools

- ❑ **Tools Supporting the DevOps Lifecycle**
- ❑ **Planning:** Jira, Trello
- ❑ **Development:** Git, GitHub
- ❑ **Building:** Maven, Gradle
- ❑ **Integrate:** Jenkins
- ❑ **Testing:** Selenium, TestNG
- ❑ **Deployment:** Docker, Kubernetes
- ❑ **Monitoring:** Prometheus, Grafana



How does a CI/CD pipeline work?

- ❑ The CI/CD pipeline consists of 3 connected methodologies:
- ❑ Continuous integration
- ❑ Continuous delivery
- ❑ Continuous deployment



CI/CD pipeline

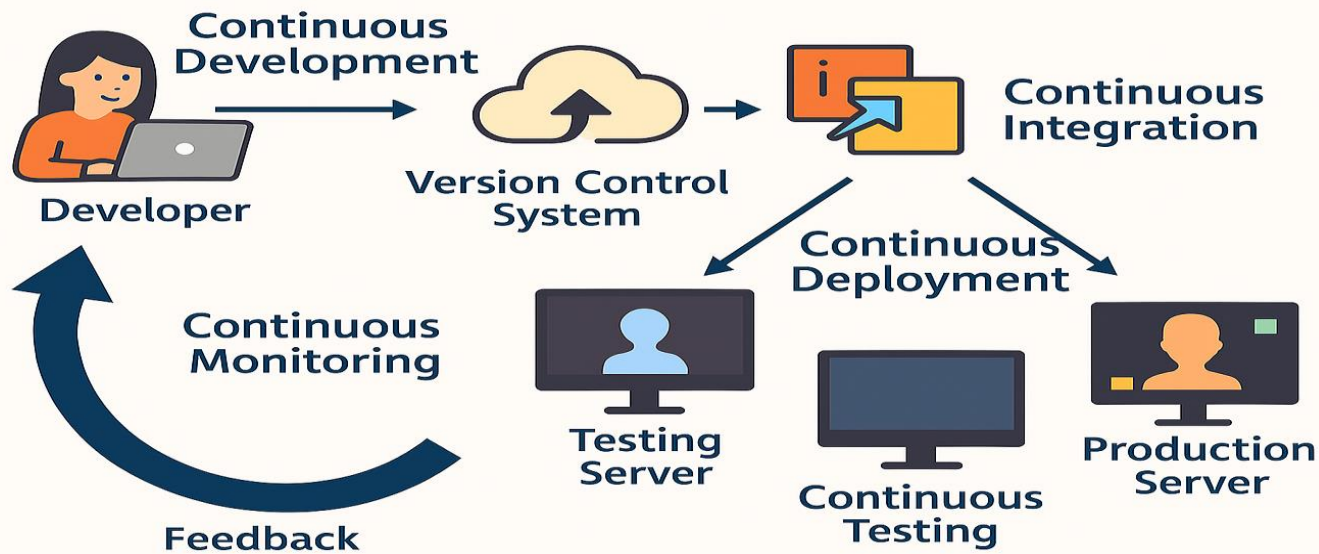
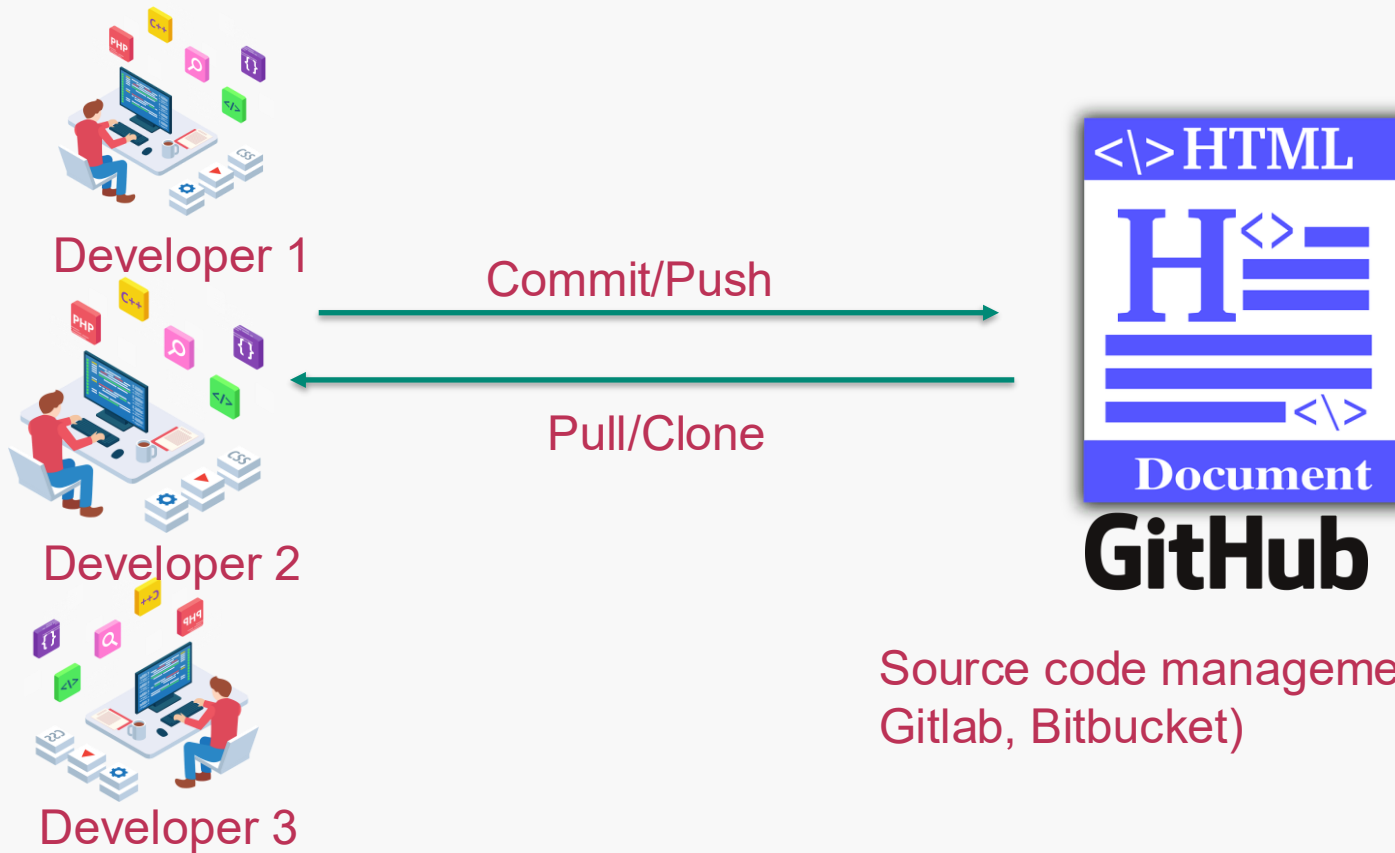


Image source: intelipaat.com

Example: Version Control



Source code management (E.g Git hub, Gitlab, Bitbucket)

Continuous Integration in DevOps

- ❑ CI automates the process of integrating code changes, ensuring they are built, tested, and stored efficiently.
- ❑ Jenkins orchestrates the entire workflow.

CI Pipeline Stages

- ❑ Source Code Management
- ❑ Build Automation
- ❑ Code Quality Check
- ❑ Artifact Storage

Jenkins automates all steps—from code fetch to artefact storage.

Example: Add "Dark Mode" Feature to the App

- ❑ Developer Tim implements the new "Dark Mode" feature and pushes the updated code to **GitHub**.
- ❑ **Jenkins** detects the push and triggers the CI pipeline.
- ❑ It uses **Maven** to compile the code and runs unit tests with JUnit to ensure the feature works as expected.
- ❑ **SonarQube** analyses the quality of the code and flags a few unused imports and recommends cleaner formatting.
- ❑ After the issues are resolved, Jenkins packages the final build and stores the updated **.jar file in Nexus**.
- ❑ The new version of the app—with Dark Mode—is now ready for deployment, fully tested and quality-checked without any manual intervention.

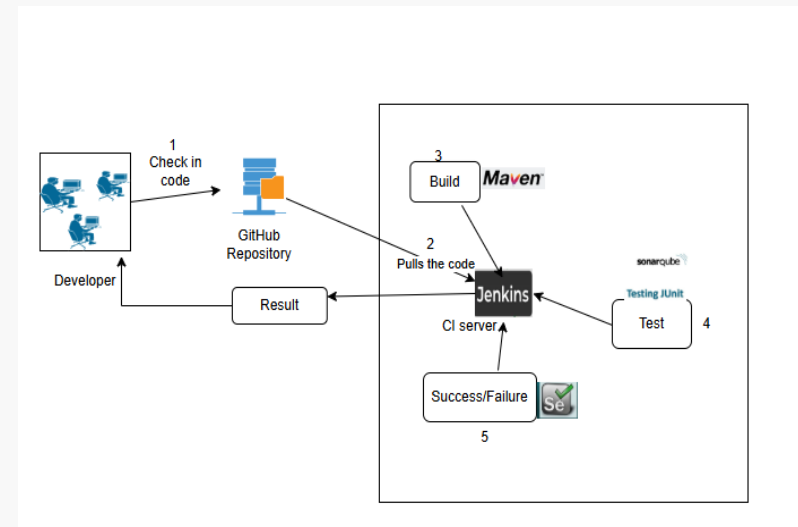


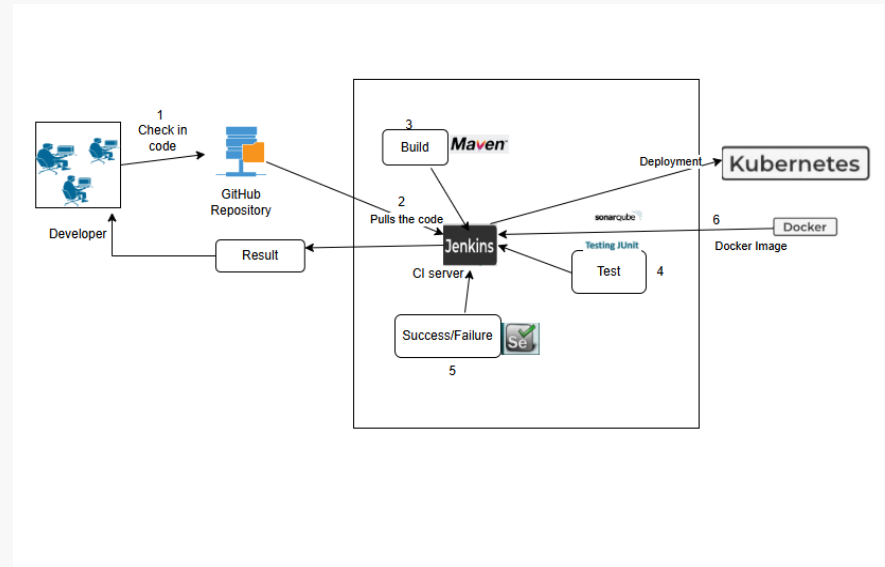
Image source: [DevOps Lifecycle - GeeksforGeeks](#)

End Result: Dark Mode feature is automatically built, tested, and packaged — ready to deploy!

From Continuous Delivery to Continuous Deployment

•**Continuous Delivery**: Every code change is automatically tested and prepared for release, but deployment to production is **manual**.

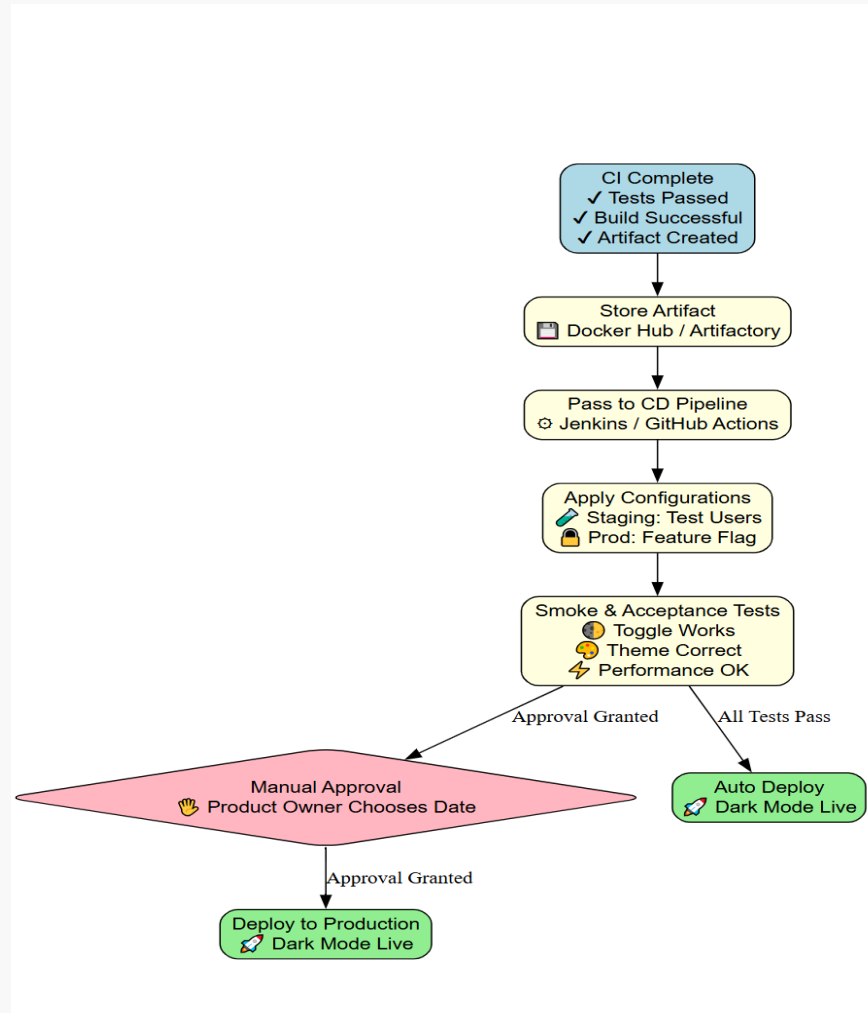
•**Continuous Deployment**: Every code change that passes all tests is **automatically** deployed to production with no manual step.



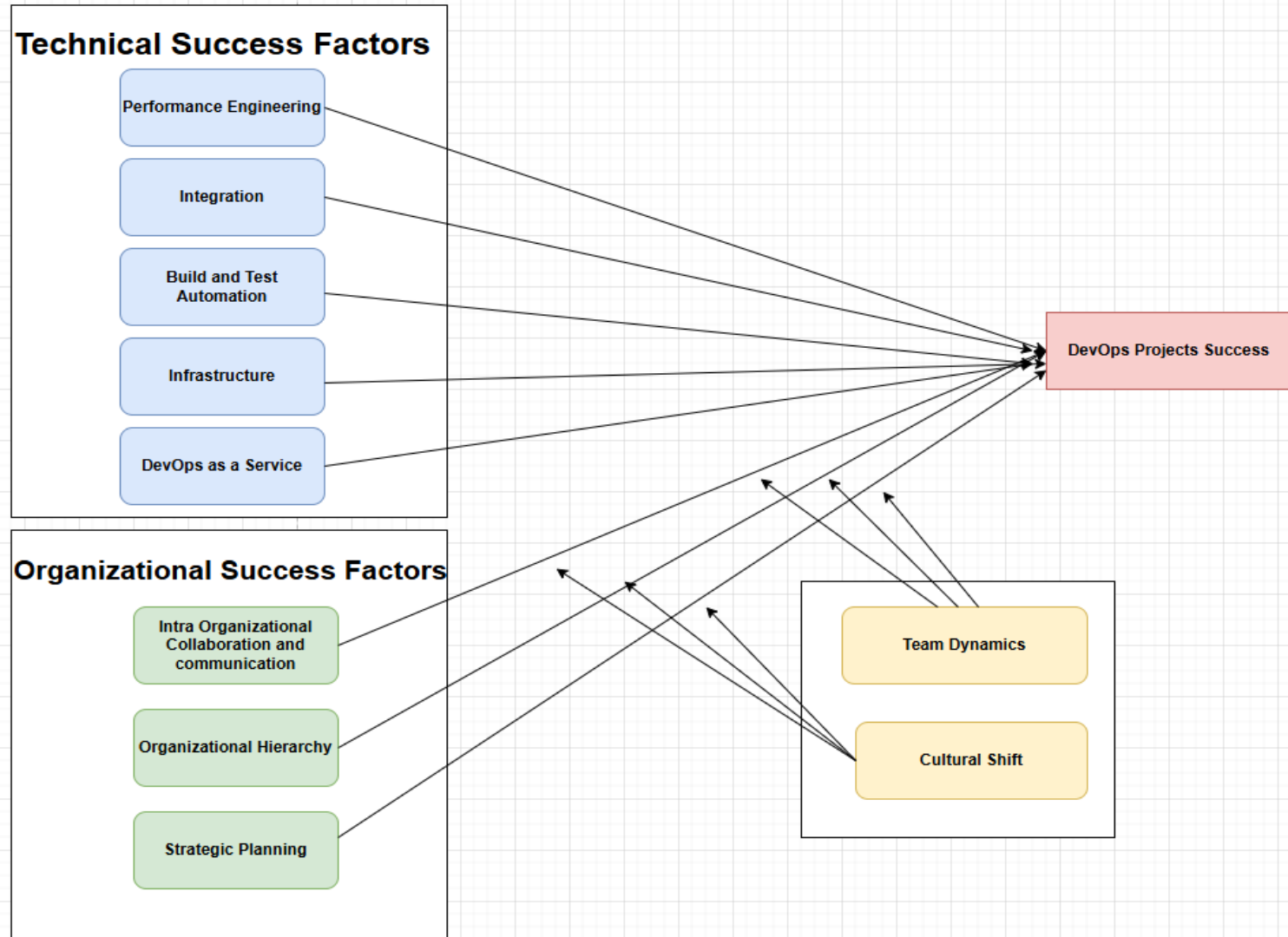
CI completed → Moving into Delivery/Deployment

Jenkins → Docker → Kubernetes (Staging → Production)

What Happens After Integration — Using the "Dark Mode" Feature



DevOps Critical Success Factors



Real-World DevOps Organizations

amazon

Walmart

Adobe

NETFLIX

Case Study: Netflix & DevOps

Delivering 270+ million subscribers uninterrupted, high-quality streaming

NETFLIX



SCALE

Global reach with millions of concurrent streams



SPEED

Hundreds of deployments daily via CI/CD



CLOUD-FIRST

AWS auto-scaling for demand spikes



RESILIENCE

Chaos Engineering ensures fault tolerance



CUSTOMER FOCUS

Real-time monitoring for quality & recommendations



KEY INSIGHT: DevOps enables Netflix to innovate fast, stay resilient, and deliver at global scale

Key Challenges in adopting DevOps

- ❑ Resistance to Change
- ❑ Lack of Executive Buy-In
- ❑ Skill Gaps
- ❑ Legacy Systems
- ❑ Tool Overload
- ❑ Fragmented Pipelines

AI in DevOps: Enhancing Development Quality

- ❑ **Planning:** NLP for requirement analysis, risk & timeline prediction
- ❑ **Coding:** AI code suggestions, bug detection, refactoring recommendations
- ❑ **CI & Testing:** Automated test generation, defect prediction, anomaly detection
- ❑ **Deployment:** Predictive deployment risk, intelligent rollback, infrastructure optimisation
- ❑ **Monitoring:** Anomaly detection, root cause analysis, predictive maintenance
- ❑ **Security:** Automated vulnerability detection, threat prediction, compliance checks

CS2SE - Software Engineering and Professional Development - Semester 1



Quiz



References

- ❑ DevOps Lifecycle – GeeksforGeeks
- ❑ <https://youtu.be/MdjhVb4y6RY?feature=shared>
- ❑ <https://youtu.be/MdjhVb4y6RY?feature=shared>
- ❑ DevOps Case Studies Compilation - DevOpsSchool.com
- ❑ 30+ Best DevOps Tools | Git, Jenkins, Selenium | Edureka
- ❑ Essential Characteristics of DevOps | Coursera
- ❑ [DevOps Case Study: Netflix and the Chaos Monkey](#)
- ❑ [Microsoft DevOps Stories | Microsoft Azure](#)
- ❑ [DevOps - Amazon Web Services \(AWS\)](#)